

STEM Voice Tutor

Demo Script & Cue Cards

A locally-hosted, voice-driven STEM tutor for children

0. Opening (30-45 seconds)

What it is: a locally-hosted, voice-driven STEM tutor for children. A child asks a question by voice or text, the system transcribes it, answers it in a friendly and encouraging way grounded in uploaded STEM textbooks, and speaks the answer back.

Why it exists: built with low-connectivity deployments in mind, developed with Sub-Saharan Africa specifically in mind. Cloud AI tutors assume cheap, reliable internet - that assumption doesn't hold everywhere. This runs fully offline: no cloud LLM, speech-to-text, or text-to-speech APIs. Once it's set up, it needs zero ongoing internet connection and zero ongoing cost per query.

Where it's headed: the next phase (not yet built) is ESP32 hardware - a physical unit with a microphone, speaker, and LCD screen that talks to this exact same backend over local WiFi. Everything shown today was built and tested with that phase specifically in mind.

1. Architecture at a Glance

ON SCREEN: (no clicking needed - context for the lecturer)

- **FastAPI** - backend serving everything.
- **Ollama** - running a local model (currently llama3.2, swappable from the admin panel).
- **faster-whisper** - for speech-to-text, runs on CPU, no GPU required.
- **Piper** - for text-to-speech, local ONNX voice models.
- **SQLite** - for all storage: documents, chunks, embeddings, interaction log, registered devices.
- **sentence-transformers + numpy** - for vector search, no external vector database needed.

Every one of those runs on the same machine as the web server. Nothing leaves the device.

2. Text Chat

ON SCREEN: Main UI -> Text Chat tab

- Ask a question that's covered by an uploaded document (e.g. "What is gravity?"). Show the answer, then click "Show sources" to reveal the exact retrieved passages, the document they came from, and a similarity score.
- Ask a natural follow-up (e.g. "how does that change on the moon?"). This demonstrates **conversation memory** - the tutor resolves "that" using the last few turns of this same session, without the learner having to repeat the whole question.
- Ask something no uploaded document covers. Point out the badge changes to "General knowledge" instead of "Grounded in documents" - the tutor never just refuses to answer. That was a deliberate design change from a stricter, zero-hallucination-only predecessor design: it's the wrong shape for a kids' tutor to go silent.

3. Voice Chat

ON SCREEN: Main UI -> Voice Chat tab

Press and hold the button, ask a question out loud, release. Same backend logic as Text Chat runs underneath - audio in, faster-whisper transcribes it, the same answer-generation path runs, Piper speaks the answer back.

- Point out the transcript box and the answer box are shown separately - if the transcript is wrong, that's visibly a hearing (STT) problem, not a thinking (LLM) problem. Useful for debugging on camera.
- This is the primary interface for the actual target users - a child who can't type yet, or is more comfortable speaking. Text Chat is the original interface this project grew from; Voice Chat is the one that matters for the deployment target.

4. Translate a Tutor Answer

ON SCREEN: click Translate under any answer, pick a language

Lets you check an answer's quality in a language you're more fluent in, without changing the tutor's actual response language. Runs through the same local model - no cloud translation API, no new dependency.

Honest caveat if asked: translation quality follows the underlying model's own multilingual ability. It's reliably good for French; noticeably weaker for Swahili with the smaller default model, since Swahili is a lower-resource language for most LLMs. Treat it as a helpful aid for spot-checking, not a certified-accurate reference.

5. Document Management

ON SCREEN: /admin/documents/management

- Upload a PDF live and show the progress indicator.

Explaining Clean / Problematic / Unknown:

- **Clean:** content matched a STEM keyword (physics, algorithm, ecosystem, etc.) and triggered no red flags.
- **Problematic:** content triggered a real check: looks like a prompt-injection attempt ("ignore previous instructions"), has too many corrupted/garbled characters, is too short to be useful, or matches an admin-configured exclude keyword.
- **Unknown:** neither of the above - not necessarily bad content, e.g. a table of contents or an index page that just doesn't happen to contain a STEM keyword.
- Click View on a document to show the real embedded PDF preview.
- Show Filter Configuration: the keyword lists are fully admin-editable and reload live - no code changes or restart needed to tune what counts as on-topic.

Good credibility story if asked about code quality: this classifier and its keyword lists were inherited from a completely different earlier project (a Swiss municipal waste-sorting assistant this codebase started as) and were fully re-audited and re-purposed for STEM content this session, including finding and fixing a real bug where "programming"/"algorithm" were being flagged for removal - backwards for a tutor where computer science is one of the four STEM pillars.

6. Interaction Log

ON SCREEN: /admin/interaction-log/view

Every question and answer is logged automatically in the background - fire-and-forget, so logging never slows down the response the learner is waiting on.

- Show rating an answer with a thumbs-up. Explain: a thumbs-up promotes that answer into a global "answer memory" - it gets embedded and added to the live search index, so a particularly good answer can help a different learner asking something similar later, without a human manually re-authoring it as a document.
- Show the Translate control here too - use case: reviewing a session that happened in Swahili or French, in English, for QA purposes.

7. Devices - Handling Multiple Physical Units

ON SCREEN: /admin/devices/view

This is the groundwork for the ESP32 hardware phase. Likely the most technical question you'll get, so know this section well.

How device identity works:

- Registering a device generates a random API key (32 bytes, cryptographically secure).
- Only a SHA-256 **hash** of that key is ever stored in the database - never the plaintext.
- The plaintext key is shown exactly once, at creation time, like a GitHub personal access token. The admin copies it into that unit's firmware immediately; it can't be recovered later.
- Every request from a device carries the key in an X-Device-Key header. The server hashes the incoming key and looks up the match - the raw key itself never appears in logs or storage.
- This means a single compromised or decommissioned unit can be revoked individually without affecting any other device, and per-device usage (last-seen time) can be tracked without ever exposing a working secret.

How this interacts with browser security (CSRF):

- Browser tabs authenticate via session-based CSRF tokens; a physical device can't do that handshake, so a valid device key bypasses the CSRF check for that request.
- A present-but-invalid device key is rejected immediately with 401, rather than silently falling through to the CSRF check - this is a **different trust model** for a different kind of client, not a weaker one. Forging a device key is exactly as hard as forging any other secret, since only its hash is ever stored.

8. Concurrency and Scale

ON SCREEN: /admin/diagnostics/view, or just talk through it

A real problem found and fixed this project: the AI call, speech-to-text, and text-to-speech were all blocking, synchronous calls sitting directly inside async request handlers. That meant while the server was busy thinking for one device, the entire server was frozen - even an unrelated health check from a different device would queue behind it.

- Fixed by moving all three off the event loop with `asyncio.to_thread`, so multiple requests can be in flight genuinely at the same time.
- **Proved it, not just claimed it:** a load-test script (`scripts/simulate_devices.py`) fired 4 concurrent AI

requests and measured about 15 seconds total wall time - versus an estimated 59 seconds if they'd been serialized one after another. A health check answered in 7 milliseconds while those 4 heavy requests were still running.

- Added a concurrency cap too (a semaphore, default 4 simultaneous voice requests) so CPU doesn't get overloaded if many devices talk at once. Excess requests get an immediate, friendly "the tutor is busy, try again in a moment" instead of hanging indefinitely.

9. Hardware Wire Protocol

ON SCREEN: /admin/protocol-test/view

No ESP32 hardware exists yet, so this page simulates what a physical unit will do: send audio with a device key and receive a compact binary response instead of the base64-encoded JSON a browser gets.

- Why it matters: an embedded device has very limited RAM and CPU. Base64 encoding costs roughly 33% more bytes to transfer and real CPU time to decode - a length-prefixed binary frame format avoids both.
- The full spec is written up in docs/PROTOCOL.md, so firmware can be developed against a fixed contract independently of the backend's own code.

10. Multi-Language Support

ON SCREEN: briefly tie back to the Translate feature

Every prompt and response string is data, not code - adding a language means dropping one YAML file into config/languages/, nothing else changes. Started with English and German (inherited from the project's earlier life), and French, Swahili, and Portuguese were added specifically because they matter for the actual Sub-Saharan Africa deployment target.

11. Closing / Roadmap

Recap in one breath: fully offline, voice-first, answers grounded in real documents with a graceful general-knowledge fallback, plus admin tooling for content curation, quality review, and multi-device management.

What's next: real ESP32 firmware. Everything shown today on the device-auth, concurrency, and binary-protocol side was built and load-tested specifically so that phase has a solid, already-proven backend to talk to the moment hardware exists.

Anticipated Q&A Cheat Sheet

Q: Why not just use a cloud LLM like GPT or Claude?

A: The deployment target has limited or costly internet access. Running fully locally means zero ongoing connectivity requirement and zero ongoing per-query cost once the device is set up.

Q: How do you identify and authenticate different physical devices?

A: Hashed, per-device API keys (see section 7) - plaintext shown once at creation, only a SHA-256 hash stored, sent via an X-Device-Key header, revocable individually.

Q: What happens if two learners ask questions at the same time?

A: Handled without blocking each other (see section 8) - blocking calls run in worker threads off the event loop, with a concurrency cap that fails fast and friendly instead of hanging when overloaded.

Q: How do you keep one learner's conversation private from another's?

A: Conversation memory is explicitly scoped per session ID, generated per browser tab/device session. This was a deliberate design decision, not an accident - a shared/global conversation memory would let one learner's chat leak into another's, which was treated as a real bug class to design out from the start.

Q: What happens if the AI model fails or is unreachable?

A: It fails soft: the learner gets a friendly "having a little trouble, try again" message, never a raw error or a crash. A related bug was found and fixed recently - a failed answer was being cached and would keep being served for up to 24 hours even after the model recovered; fixed so failures are never cached.

Q: How do you know an uploaded document is relevant or appropriate?

A: The Clean / Problematic / Unknown classifier (section 5) flags real problems automatically - prompt-injection attempts, corrupted encoding, too-short content - and an admin makes the final call on anything flagged or Unknown.

Q: Is any data sent to the cloud?

A: No. The LLM, speech-to-text, text-to-speech, storage, and vector search all run locally on the same machine. Nothing about a learner's question or the tutor's answer leaves the device.

Q: Why SQLite instead of a client-server database like Postgres?

A: Matches the deployment constraint directly: a single local device with zero setup, no separate database server to install, configure, or keep running.

Q: How would this scale to a classroom with many devices at once?

A: That's exactly what section 8's concurrency work and load test were built to answer ahead of having real hardware to test against - the semaphore cap is the explicit, tunable answer to "how many at once," and it's configurable via an environment variable without a code change.